



MANIPAL

ACADEMY *of* HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Cryptology – Lab 3

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cyber Security

25/08/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES

MANIPAL

(A constituent unit of MAHE, Manipal)

1. Debug the following programs. Identify the type of error in each case.

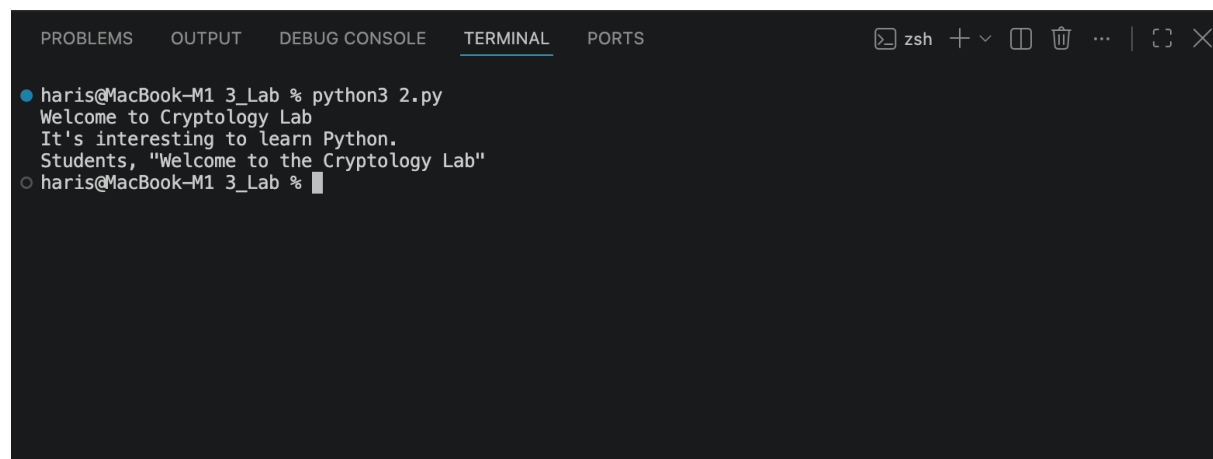
```
print("Welcome to Cryptology Lab")
message = "Python Programming"
print(mesage)
num = input("Enter a number: ")
result = num + 10
print(result)
```

```
print("Welcome to Cryptology Lab")
# SyntaxError: unterminated string literal
message = "Python Programming"
print(mesage)
# NameError: name 'mesage' is not defined.
num = input("Enter a number: ")
# TypeError: can only concatenate str (not "int") to str
result = num + 10
print(result)
```

2. Write Python statements to display the following exactly:

Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab".

```
print("Welcome to Cryptology Lab")
print("It's interesting to learn Python.")
print("Students, \"Welcome to the Cryptology Lab\"")
```



The screenshot shows a terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
zsh + v [ ] [ ] ... | [ ] [ ] X

● haris@MacBook-M1 3_Lab % python3 2.py
Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab"
○ haris@MacBook-M1 3_Lab %
```

3. Read two numbers from the user and perform:

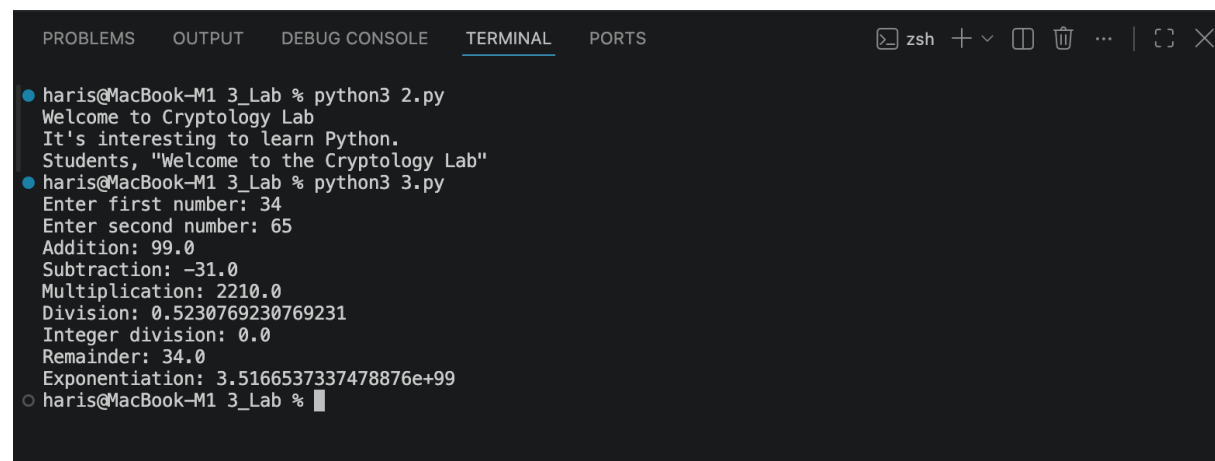
Addition
Subtraction
Multiplication

Division
Integer division
Remainder
Exponentiation

Display each result with an appropriate message.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Integer division:", a // b)
print("Remainder:", a % b)
print("Exponentiation:", a ** b)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● haris@MacBook-M1 3_Lab % python3 2.py
Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab"
● haris@MacBook-M1 3_Lab % python3 3.py
Enter first number: 34
Enter second number: 65
Addition: 99.0
Subtraction: -31.0
Multiplication: 2210.0
Division: 0.5230769230769231
Integer division: 0.0
Remainder: 34.0
Exponentiation: 3.5166537337478876e+99
○ haris@MacBook-M1 3_Lab %
```

4. Read a floating-point number and display:

its data type;
its absolute value;
the value rounded to two decimal places.

```
num = float(input("Enter a floating-point number: "))

print("Data type:", type(num))
print("Absolute value:", abs(num))
print("Rounded to 2 decimal places:", round(num, 2))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 4.py
Enter a floating-point number: 45.6
Data type: <class 'float'>
Absolute value: 45.6
Rounded to 2 decimal places: 45.6
haris@MacBook-M1 3_Lab %
```

Part B — Data Types and Collections (~25 min)

5. Create variables containing examples of:

int, float, complex, str, list, tuple, set, and dictionary.

Use Python to display the value and data type of each variable.

```
a = 10
b = 3.14
c = 2 + 5j
d = "Hello"
e = [1, 2, 3]
f = (4, 5, 6)
g = {7, 8, 9}
h = {"name": "John", "age": 25}

variables = [a, b, c, d, e, f, g, h]

for x in variables:
    print("Value:", x, "| Type:", type(x))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 5.py
Value: 10 | Type: <class 'int'>
Value: 3.14 | Type: <class 'float'>
Value: (2+5j) | Type: <class 'complex'>
Value: Hello | Type: <class 'str'>
Value: [1, 2, 3] | Type: <class 'list'>
Value: (4, 5, 6) | Type: <class 'tuple'>
Value: {8, 9, 7} | Type: <class 'set'>
Value: {'name': 'John', 'age': 25} | Type: <class 'dict'>
haris@MacBook-M1 3_Lab %
```

6. Demonstrate experimentally that:

Strings are immutable, whereas lists are mutable.

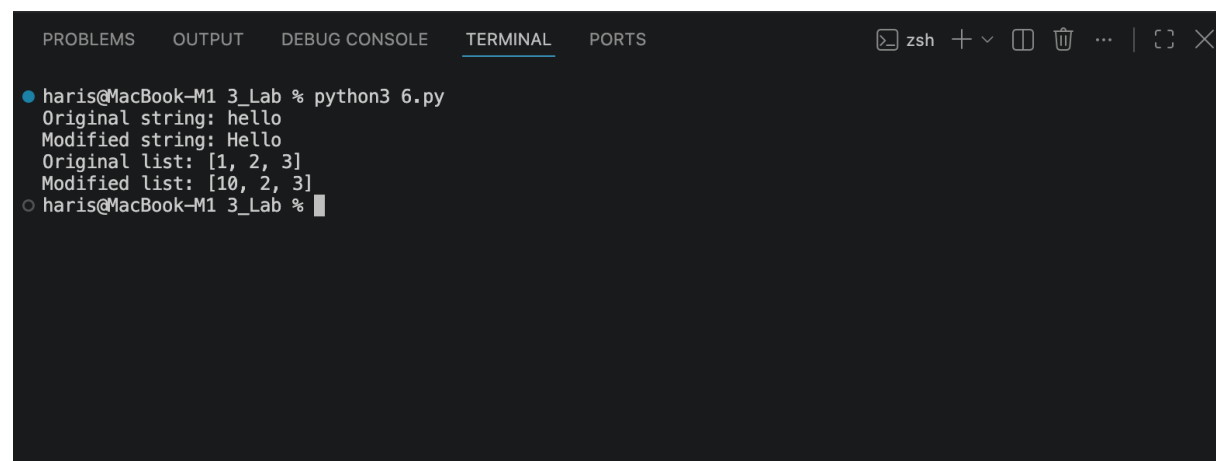
Include the Python statements you used and explain the observed result.

```
# String (immutable)
s = "hello"
print("Original string:", s)

# s[0] = "H"    # This will cause an error
s = "Hello"     # Reassignment creates a new string
print("Modified string:", s)

# List (mutable)
lst = [1, 2, 3]
print("Original list:", lst)

lst[0] = 10
print("Modified list:", lst)
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● haris@MacBook-M1 3_Lab % python3 6.py
Original string: hello
Modified string: Hello
Original list: [1, 2, 3]
Modified list: [10, 2, 3]
○ haris@MacBook-M1 3_Lab %
```

7. Create a list containing five prime numbers. Perform the following:

- display the first three elements;
- display the last element;
- append another prime number;
- extend the list with two more prime numbers;
- display the total number of elements.

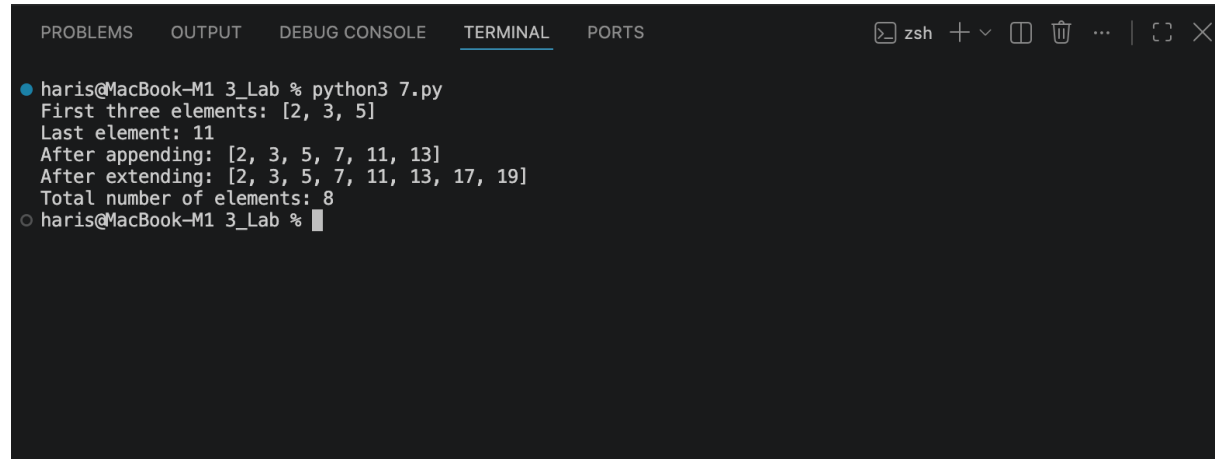
```
primes = [2, 3, 5, 7, 11]

print("First three elements:", primes[:3])
print("Last element:", primes[-1])

primes.append(13)
print("After appending:", primes)
```

```
primes.extend([17, 19])
print("After extending:", primes)

print("Total number of elements:", len(primes))
```



```
haris@MacBook-M1 3_Lab % python3 7.py
First three elements: [2, 3, 5]
Last element: 11
After appending: [2, 3, 5, 7, 11, 13]
After extending: [2, 3, 5, 7, 11, 13, 17, 19]
Total number of elements: 8
haris@MacBook-M1 3_Lab %
```

Part C — String Operations

For Questions 8–12, use:

```
message = "CRYPTOLOGY USING PYTHON"
```

8. Display:

- length of the string;
- first five characters;
- last six characters;
- characters from positions 3 to 8;
- string in reverse order.

```
message = "CRYPTOLOGY USING PYTHON"

print("Length:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
print("Characters from positions 3 to 8:", message[2:8])
print("Reverse:", message[::-1])
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● haris@MacBook-M1 3_Lab % python3 8.py
Length: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from positions 3 to 8: YPTOLO
Reverse: NOHTYP GNISU YGOLOTPYRC
○ haris@MacBook-M1 3_Lab %
```

9. Perform the following:

convert to lowercase;
convert back to uppercase;
check whether "PYTHON" exists in the string;
replace "PYTHON" with "PROGRAMMING".

```
message = "CRYPTOLOGY USING PYTHON"

lower_msg = message.lower()
upper_msg = lower_msg.upper()
has_python = "PYTHON" in message
new_msg = message.replace("PYTHON", "PROGRAMMING")

print("Lowercase:", lower_msg)
print("Uppercase:", upper_msg)
print('Contains "PYTHON":', has_python)
print("Replaced string:", new_msg)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● haris@MacBook-M1 3_Lab % python3 9.py
Lowercase: cryptology using python
Uppercase: CRYPTOLOGY USING PYTHON
Contains "PYTHON": True
Replaced string: CRYPTOLOGY USING PROGRAMMING
○ haris@MacBook-M1 3_Lab %
```

10. Remove the spaces from the string and display the resulting string.

```
message = "CRYPTOLOGY USING PYTHON"

result = message.replace(" ", "")
print(result)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 9.py
Lowercase: cryptology using python
Uppercase: CRYPTOLOGY USING PYTHON
Contains "PYTHON": True
Replaced string: CRYPTOLOGY USING PROGRAMMING
haris@MacBook-M1 3_Lab %
```

11. Using a loop, display every character and its corresponding Unicode/ASCII value using `ord()`.

Example:

C -> 67
R -> 82
Y -> 89
...

```
message = "CRYPTOLOGY USING PYTHON"

for ch in message:
    print(ch, "->", ord(ch))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 11.py
C -> 67
R -> 82
Y -> 89
P -> 80
T -> 84
O -> 79
L -> 76
O -> 79
G -> 71
Y -> 89
- -> 32
U -> 85
S -> 83
I -> 73
N -> 78
G -> 71
- -> 32
P -> 80
Y -> 89
T -> 84
H -> 72
O -> 79
N -> 78
haris@MacBook-M1 3_Lab %
```

12. Use `chr()` to convert the following values into characters:

65, 67, 73, 80, 72, 69, 82

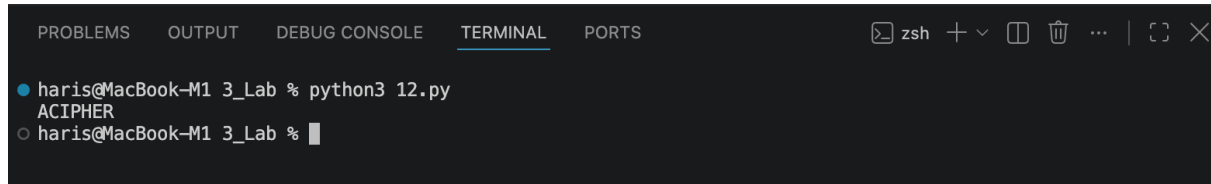
Combine the resulting characters and identify the word.

```
values = [65, 67, 73, 80, 72, 69, 82]
```



```
word = ""
for v in values:
    word += chr(v)

print(word)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 12.py
ACIPHER
haris@MacBook-M1 3_Lab %
```

Part D — Text Processing for Cryptology

13. Plaintext preprocessing

Read a message from the user and:

- remove leading/trailing spaces;
- convert it to uppercase;
- remove spaces between words.

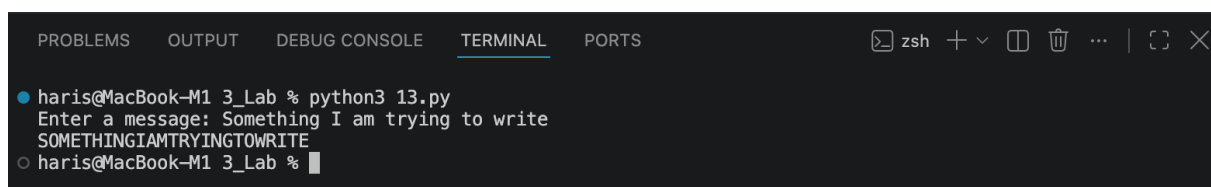
Example:

Input : meet me tomorrow
Output: MEETMETOMORROW

```
message = input("Enter a message: ")

processed = message.strip().upper().replace(" ", "")

print(processed)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 13.py
Enter a message: Something I am trying to write
SOMETHINGIAMTRYINGTOWRITE
haris@MacBook-M1 3_Lab %
```

14. Character grouping

Read a string and divide it into blocks of five characters.

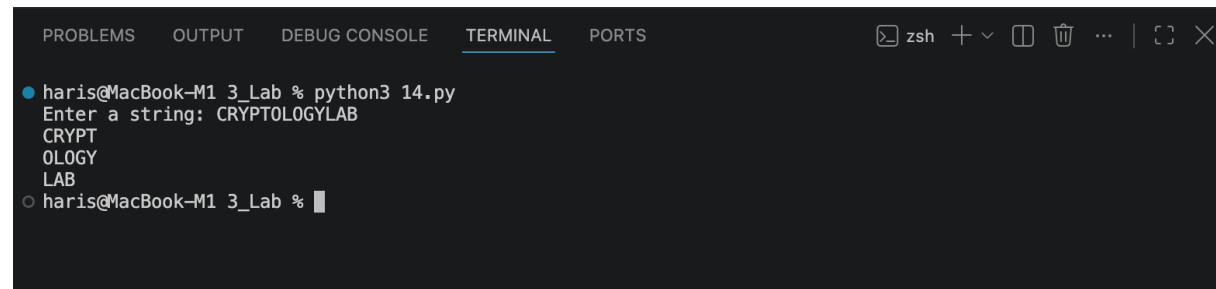
Example:

Input: CRYPTOLOGYLAB

Output:
CRYPT
OLOGY
LAB

```
text = input("Enter a string: ")

for i in range(0, len(text), 5):
    print(text[i:i+5])
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
zsh + v [ ] [ ] ... | [ ] [ ] X

● haris@MacBook-M1 3_Lab % python3 14.py
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LAB
○ haris@MacBook-M1 3_Lab %
```

15. Padding

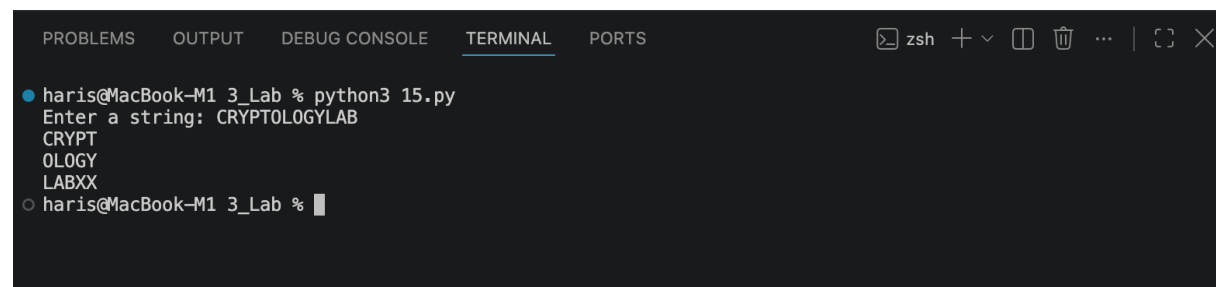
Modify Question 14 so that an incomplete final block is padded with X.

Input: CRYPTOLOGYLAB

Output:
CRYPT
OLOGY
LABXX

```
text = input("Enter a string: ")

for i in range(0, len(text), 5):
    block = text[i:i+5]
    if len(block) < 5:
        block = block + "X" * (5 - len(block))
    print(block)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
zsh + v [ ] [ ] ... | [ ] [ ] X

● haris@MacBook-M1 3_Lab % python3 15.py
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LABXX
○ haris@MacBook-M1 3_Lab %
```

16. Character frequency

For the following text:

```
text = "CRYPTOLOGYISINTERESTING"
```

write a program to calculate the frequency of each character.

Then identify the most frequently occurring character.

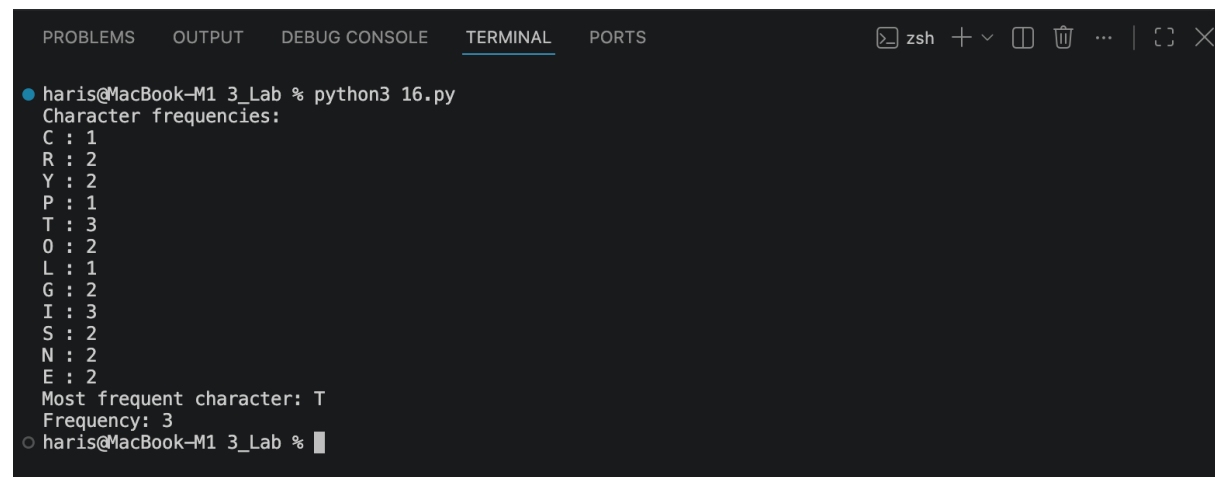
```
text = "CRYPTOLOGYISINTERESTING"

freq = {}

for ch in text:
    if ch in freq:
        freq[ch] += 1
    else:
        freq[ch] = 1

print("Character frequencies:")
for ch in freq:
    print(ch, ":", freq[ch])

most_char = max(freq, key=freq.get)
print("Most frequent character:", most_char)
print("Frequency:", freq[most_char])
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● haris@MacBook-M1 3_Lab % python3 16.py
Character frequencies:
C : 1
R : 2
Y : 2
P : 1
T : 3
O : 2
L : 1
G : 2
I : 3
S : 2
N : 2
E : 2
Most frequent character: T
Frequency: 3
○ haris@MacBook-M1 3_Lab %
```

17. Percentage frequency

Extend Question 16 to calculate:

Percentage Frequency=

Total Number of Characters/ Frequency of Character $\times 100$

Display the percentage rounded to two decimal places.

```

text = "CRYPTOLOGYISINTERESTING"

freq = {}

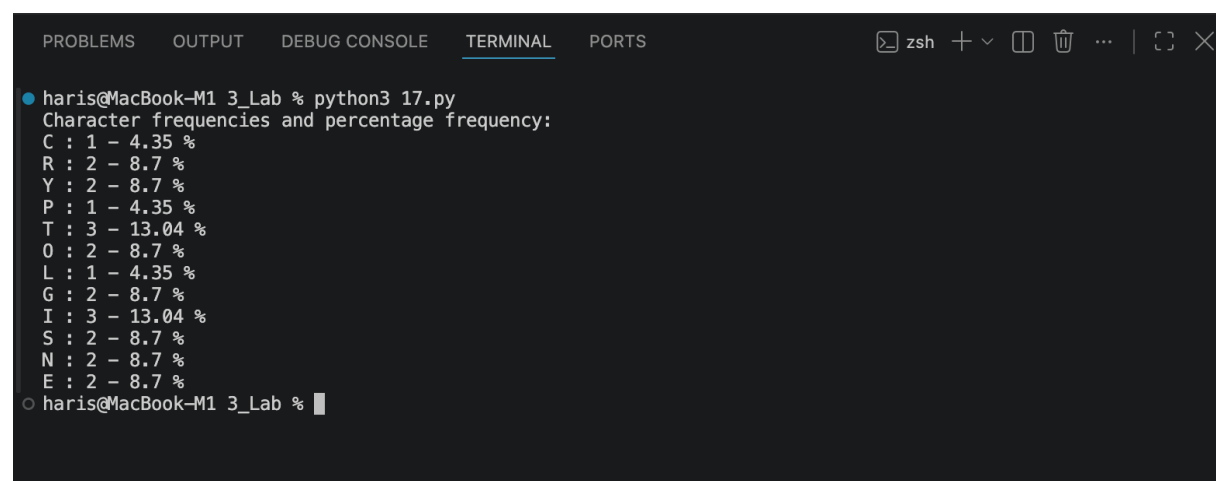
for ch in text:
    if ch in freq:
        freq[ch] += 1
    else:
        freq[ch] = 1

total = len(text)

print("Character frequencies and percentage frequency:")

for ch in freq:
    percentage = (freq[ch] / total) * 100
    print(ch, ":", freq[ch], "-", round(percentage, 2), "%")

```



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
zsh + v [ ] [ ] ... | [ ] [ ] X

● haris@MacBook-M1 3_Lab % python3 17.py
Character frequencies and percentage frequency:
C : 1 - 4.35 %
R : 2 - 8.7 %
Y : 2 - 8.7 %
P : 1 - 4.35 %
T : 3 - 13.04 %
O : 2 - 8.7 %
L : 1 - 4.35 %
G : 2 - 8.7 %
I : 3 - 13.04 %
S : 2 - 8.7 %
N : 2 - 8.7 %
E : 2 - 8.7 %
○ haris@MacBook-M1 3_Lab %

```

Part E — Mathematical Foundations for Cryptology

18. Modular arithmetic

Use Python to evaluate:

$29 \bmod 26$
 $55 \bmod 26$
 $78 \bmod 26$
 $-3 \bmod 26$
 $-29 \bmod 26$

Then write a program that accepts an integer and calculates the integer modulo 26.

```

print("29 mod 26 =", 29 % 26)
print("55 mod 26 =", 55 % 26)

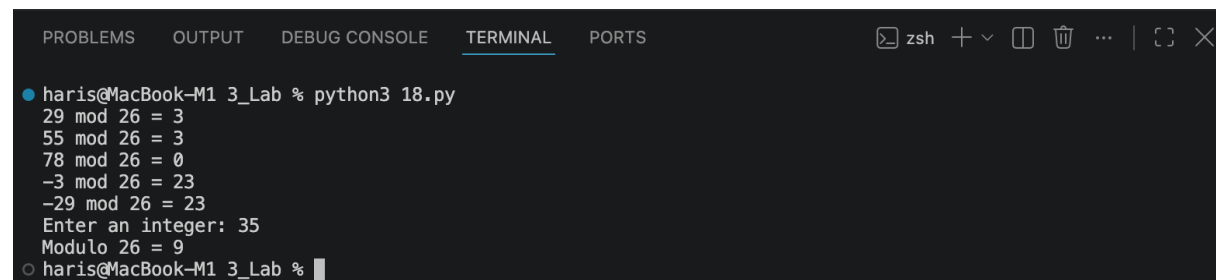
```

```

print("78 mod 26 =", 78 % 26)
print("-3 mod 26 =", -3 % 26)
print("-29 mod 26 =", -29 % 26)

num = int(input("Enter an integer: "))
print("Modulo 26 =", num % 26)

```



```

haris@MacBook-M1 3_Lab % python3 18.py
29 mod 26 = 3
55 mod 26 = 3
78 mod 26 = 0
-3 mod 26 = 23
-29 mod 26 = 23
Enter an integer: 35
Modulo 26 = 9
haris@MacBook-M1 3_Lab %

```

19. Alphabet representation

Using

A=0, B=1,...,Z=25

write a program that accepts an uppercase character and displays its numerical representation.

Example:

Enter character: M

M -> 12

Also perform the reverse conversion:

Enter number: 12

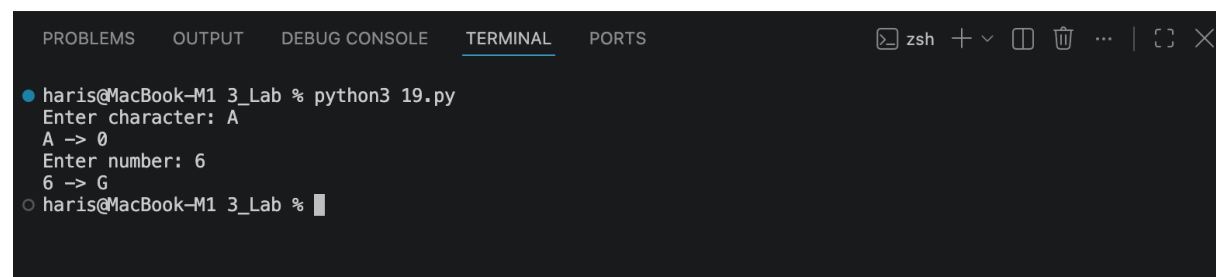
12 -> M

```

ch = input("Enter character: ").upper()
print(ch, "->", ord(ch) - ord("A"))

num = int(input("Enter number: "))
print(num, "->", chr(num + ord("A")))

```



```

haris@MacBook-M1 3_Lab % python3 19.py
Enter character: A
A -> 0
Enter number: 6
6 -> G
haris@MacBook-M1 3_Lab %

```

20. Prime numbers

Write a program to:

determine whether a given number is prime;
display all prime numbers within a range specified by the user.

```
num = int(input("Enter a number: "))

if num < 2:
    print(num, "is not prime")
else:
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        print(num, "is prime")
    else:
        print(num, "is not prime")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

print("Prime numbers:")
for n in range(start, end + 1):
    if n >= 2:
        prime = True
        for i in range(2, n):
            if n % i == 0:
                prime = False
                break
        if prime:
            print(n)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
haris@MacBook-M1 3_Lab % python3 20.py
Enter a number: 43
43 is prime
Enter start of range: 2
Enter end of range: 34
Prime numbers:
2
3
5
7
11
13
17
19
23
29
31
haris@MacBook-M1 3_Lab %
```

21. GCD and coprimes

Read two integers from the user. Find their GCD and determine whether they are coprime.

Test your program with:

15 and 26

18 and 24

17 and 26

```
a = int(input("Enter first integer: "))
b = int(input("Enter second integer: "))

x, y = a, b
while y != 0:
    x, y = y, x % y

print("GCD =", x)

if x == 1:
    print("They are coprime")
else:
    print("They are not coprime")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 21.py
Enter first integer: 15
Enter second integer: 26
GCD = 1
They are coprime
haris@MacBook-M1 3_Lab % python3 21.py
Enter first integer: 18
Enter second integer: 24
GCD = 6
They are not coprime
haris@MacBook-M1 3_Lab % python3 21.py
Enter first integer: 17
Enter second integer: 26
GCD = 1
They are coprime
haris@MacBook-M1 3_Lab %
```

22. Write a program to get the multiplicative inverse of an integer b.

```
b = int(input("Enter b: "))

inverse = None
for x in range(1, 26):
    if (b * x) % 26 == 1:
        inverse = x
        break

if inverse is None:
    print("No multiplicative inverse exists mod 26")
else:
    print("Multiplicative inverse of", b, "is", inverse)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
haris@MacBook-M1 3_Lab % python3 22.py
Enter b: 345
Multiplicative inverse of 345 is 15
haris@MacBook-M1 3_Lab % python3 22.py
Enter b: 9
Multiplicative inverse of 9 is 3
haris@MacBook-M1 3_Lab % python3 22.py
Enter b: 15
Multiplicative inverse of 15 is 7
haris@MacBook-M1 3_Lab %
```

Part F — Challenge / If Time Permits (~15 min)

Bonus:

23. WAP in python to implement client and server communication.


```
# client.py
import socket

host = "127.0.0.1"
port = 12345

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((host, port))

client.send("Hello from client".encode())

reply = client.recv(1024).decode()
print("Server says:", reply)

client.close()
```

```
# server.py
import socket

host = "127.0.0.1"
port = 12345

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen(1)

print("Server is waiting for a connection...")
conn, addr = server.accept()
print("Connected by:", addr)

message = conn.recv(1024).decode()
print("Client says:", message)

conn.send("Hello from server".encode())

conn.close()
server.close()
```